

Scientific Programming in Python

Dr. Mohammed Hasanuzzaman

Room J102, Melbourne Building (Office)

e-mail:mohammed.hasanuzzaman@mtu.ie

September 2021

- **Educational Background**

- **PhD**, French National Centre for Scientific Research (CNRS), Ecole Nationale Supérieure d'Ingénieurs de Caen – ENSICAEN and University of Caen Normandie, France

- **Key Experiences**

- **Assistant Professor**, IIT Jammu, India
- **Postdoctoral Researcher**, ADAPT Centre, Dublin, Ireland
- **NLP Researcher**, BrandCrafter Ltd., Hong Kong
- **Officer-on-Special Duty (OSD)**, Govt. of West Bengal, India

- **Research Interests**

- **Natural Language Processing (NLP)**

- Information Extraction and Information Retrieval,
 - Knowledge Representation
 - Machine Translation,
 - Psycholinguistics,
 - Social Media Analytics
 - Machine Learning (deep learning) applications to NLP

- **For more information, please visit**

- <https://mohammedhasanuzzaman.github.io>**

Acknowledgments

I am grateful to Dr. Diarmuid and Dr. Ted Scully who shared with me the following material.

Module Overview

Module Name	Scientific Prog. In Python
Module Code	COMP8060
Module CRN	13896
Module URL	https://courses.cit.ie/index.cfm/page/module/moduleId/13896
Credits	5 Credits
Lecturer	Dr. Mohammed Hasanuzzaman
Contact Information	Mohammed.Hasanuzzaman@cit.ie

Module Description

General

- In this module, the learner will use the Python programming language to manipulate, manage and process data.
- More specifically, statistical and numerical libraries will be applied to analyse and manipulate complex data sets.
- The learner will also use Linux commands to perform basic system and file operations.

Module Description

Learning Outcomes

- Apply Linux commands to perform basic system and file operations
- Implement a program to solve data-driven problems by applying standard programming concepts.
- Utilise standard programming libraries and their associated functionality to perform analysis of datasets.
- Apply programming techniques to clean, transform and query data.
- Illustrate data sets using appropriate visualisation techniques.

Module Description

Indicative content

Linux

- Introduction to Linux file system and commands. Command line overview. Directory navigation, creation of directories and files, manipulating files, permissions, regular expression, process management, editors.

Programming Concepts:

- Categories of programming language, their typical application, programming in an analytical and scientific context, Language, Syntax, error checking and debugging, variables and basic data types, lists, dictionaries and sets. Processing data structures: Conditionals and Loops. Efficient code structure: functions, modules, packages and files. Engineering code: objects, classes, and Object Oriented Programming (OOP). Introduction to version control, repositories, branching, commits and merging.

Data Array Manipulation

- Overview of standard programming libraries for numerical computation. Creating multi-dimensional arrays. Performing operations such as indexing, slicing, boolean indexing, fancy indexing, building queries, transposing and applying conditional logic to arrays.

Module Description

Indicative content

Numerical and Scientific Computing:

- Use of open source numerical and scientific libraries for: performing mathematical and statistical routines, filtering, sorting, ranking, function application and mapping. Data loading, storage and different file formats. Data wrangling in the form of cleaning, transforming, merging and reshaping data. Data aggregation and grouping operations.

Visualisation

- Visualisation and interpretation of data using 2D and 3D plots such as libraries such as Matplotlib and Seaborn. Line graphs, bar plots, violin plots, KDE plots, heatmaps, etc.

Python Packages

- NumPy, SciPy, Matplotlib, Seaborn, Pandas

Assessment (100% CA)

- Practical Skills Evaluation (60%)
 - Week 3 (1hr MCQ) – 20%
 - Week 11 (2hr Lab Assessment) – 40% (open book)
- Project Work (40%) (Due Semester End)

Python Module Content

- Introduction to Python Programming Concepts and Fundamentals
 - Decision Structures, Iteration, Functions, List, Dictionaries, Tuples, Files, Objects
- Development of Python Programs for Solving Data Driven Problems
 - Numpy, Scipy, Pandas, Mathplotlib



(Fundamentals are Building Blocks)

Useful Textbooks for Python

- Eli Bressert, SciPy and NumPy: An Overview for Developers, O' Reilly Media [ISBN:978-1449305468]
- Wes McKinney, Python for Data Analysis, O' Reilly Media [ISBN: 978-1449319793] (Ebook in the library)
- Hans Petter Langtangen,, A Primer on Scientific Programming with Python [ISBN:9783642183652]
- Philipp K. Janert, A hands-on guide for programmers and data scientists Larger Cover Data Analysis with Open Source Tools, O' Reilly [ISBN: ISBN 10: 0-596-80235-8]

Useful Websites

- Python Documentation
 - <http://www.python.org/doc/>
- Python E-Book
 - <http://learnpythonthehardway.org/book/>
- New Boston Python Video Tutorials
 - <https://www.youtube.com/playlist?list=PLEA1FEF17E1E5C0DA>
- Python Data Analysis Library
 - <http://pandas.pydata.org/>
- Python Scientific Library
 - <http://www.scipy.org/>
- Python Numerical Library
 - <http://www.numpy.org/>
- Continuum Analytics (Download Anaconda)
 - <http://continuum.io/downloads>

LINUX

Linux History

Free Software and GNU

- 60s-80s. Economic model moved from expensive computers, software an add-on, to cheap computers where software became an additional income source for producers.
- As computers became cheaper and more numerous in the late 1970s, producers of computers began to distribute their software under licenses and in forms that restricted its users' abilities to use, redistribute, or modify the code.
 - In other words there was 'value' in the software itself.
- By the early 1980s, restrictive software licenses had become the norm.

Linux History

Free Software and GNU

- To fight against this loss of the freedoms that software users and developers had up until that point enjoyed, Richard Stallman - a programmer at MIT, articulated a vision for a community that developed liberated code—in his words, “free software.”

Linux History

Free Software and GNU

- He defined free software as software that had the following four characteristics:
 - The freedom to run the program for any purpose (freedom 0)
 - The freedom to study how the program works and adapt it to your needs (freedom 1)
 - The freedom to redistribute copies so you can help your neighbor (freedom 2)
 - The freedom to improve the program and release your improvements to the public so that the whole community benefit (freedom 3)

Linux History

Free Software and GNU

- In addition to releasing this definition of free software, Stallman began a project with the goal of creating a completely free OS to replace the then popular UNIX.
- In 1984, Stallman announced this project and called it GNU—a joke in the form of a recursive acronym for “GNU’s Not UNIX.”
 - Unix developed by Bell Labs in 70s, moving from assembly language based OS to higher-level C language, resulting it in being more portable to other computing platforms.
 - But was commercial software.

Linux History

Linux

- By the early 1990s, Stallman and a collection of other programmers working on GNU had developed a near-complete OS that could be freely shared.
- They were, however, missing a final essential piece in the form of a **kernel**— a complex system command processor that lies at the center of any OS.
- In 1991, Linus Torvalds wrote an early version of just such a kernel, released it under a free license, and called it Linux.

Linux History

Linux

- The Linux kernel was paired with the GNU project's development tools and OS and with the graphical windowing system called X.
- With this pairing, a completely free OS was born—free both in terms of price and in Stallman's terms of freedom.
- All systems referred to as Linux today are, in fact, built on the work of this collaboration.
- Technically, the term Linux refers only to the kernel.

Linux History

Free Software

- Free software is built in an open way
 - *Anyone can contribute to software* by looking through the code, finding bugs, and fixing them.
- Unintended (nice!) side-effect:
- Because software ended up being examined by larger numbers of programmers
 - free software was higher in quality, performed better, and offered more features
 - In many situations, the development model behind free software led to software that was *inherently better* than proprietary alternatives!

Linux History

Why is Free Software called Open Source?

- The term *free software* was problematic because:
 - It was highly ambiguous—the English word free means both gratis, or at no cost (e.g., as in “free beer”) and liberated in the sense of freedom (e.g., as in “free speech”).
 - There was a feeling, that all the talk of freedom was scaring off the business executives and decision makers whom the free software movement needed to impress in order to succeed.

Linux History

Why is Free Software called Open Source?

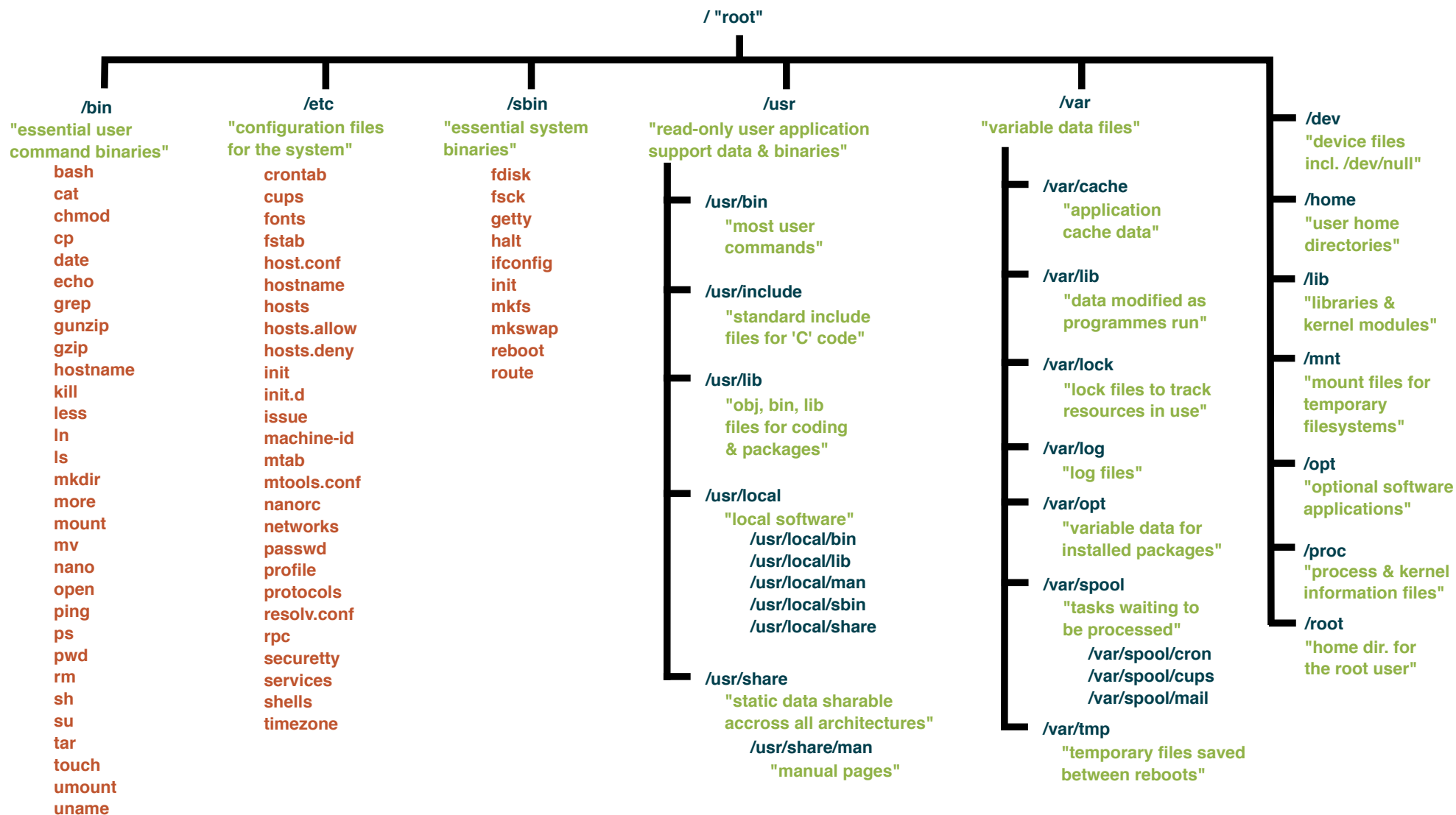
- To tackle these problems, a group of free software developers and leaders coined a new phrase—***Open source***— and created a new organization called the Open Source Initiative.
- The group set at its core a definition of open source software that overlapped completely and exclusively both with Stallman's four-part definition of free software and with other community definitions that were also based on Stallman's.

Linux History

Why is Free Software called Open Source?

- You could say, one group is focused on freedom, while the other is focused on pragmatics.
- *Free software* is most accurately described as a social movement.
- Open source is a development methodology. However, the two groups have no trouble working on projects hand in hand.

File System Hierarchy



Core Commands

- How to move around Linux system
- Run commands as root / switch between root and user
- View directory/file information
- Check running processes
- Change permissions/ownership for files

Moving around Linux system

<code>pwd</code>	Show current directory
<code>cd [option] [directory]</code>	
<code>cd</code>	Move to user home directory
<code>cd /</code>	Move to root directory
<code>cd <i>dirname</i></code>	Move to subdirectory <i>dirname</i> of current directory
<code>cd ~/<i>dirname</i></code>	Move to subdirectory <i>dirname</i> of home directory
<code>cd /<i>dirname</i></code>	Move to subdirectory <i>dirname</i> of root directory
<code>cd ../</code>	Move to parent directory (relative path)
<code>cd -</code>	Move to previous directory (v. useful when jumping back and forth between 2 directories)

Moving around Linux system

Go to home directory

Go to root directory

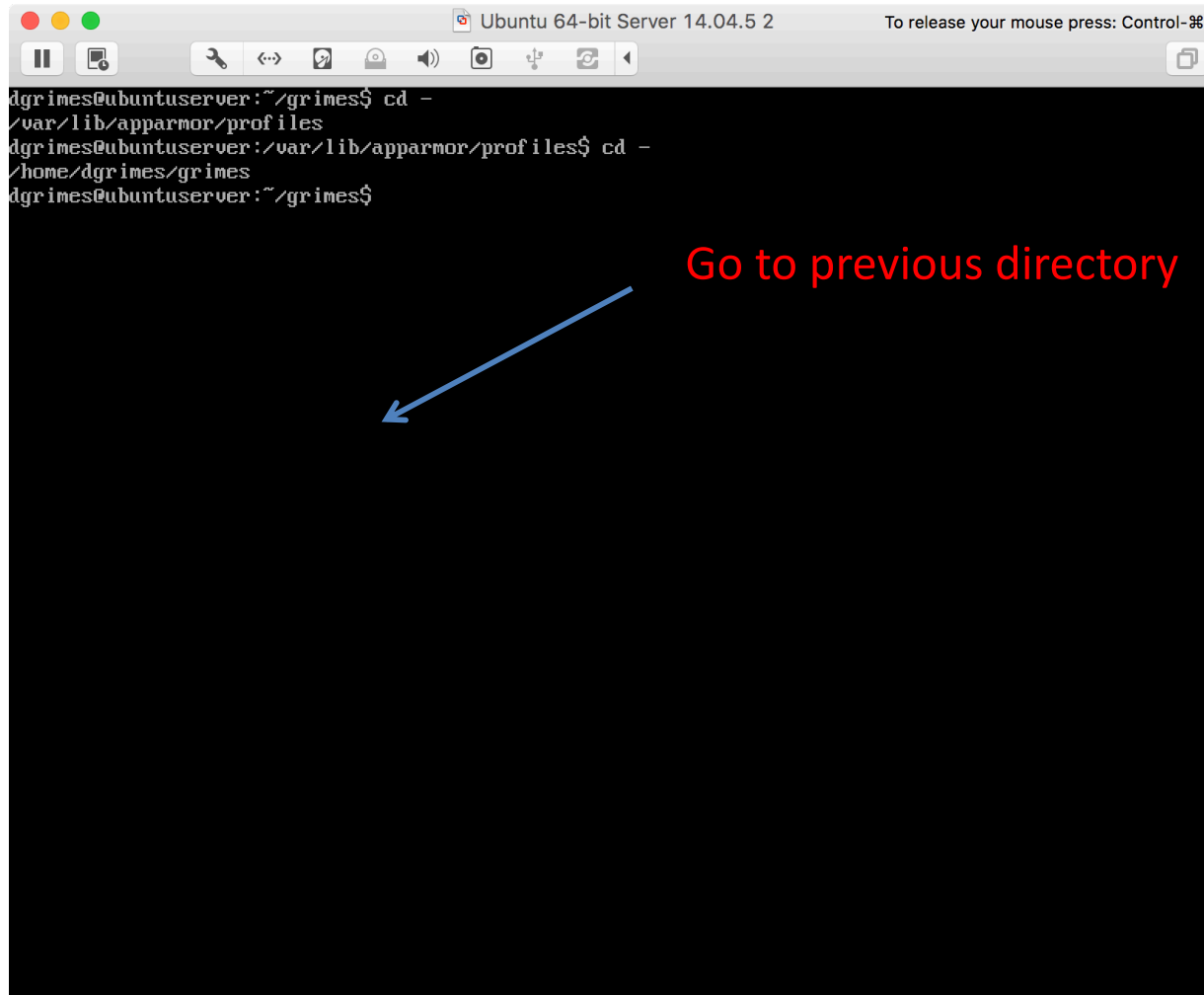
Go to subdirectory *grimes* of home directory

```
dgrimes@ubuntuuserver:~/grimes$ cd
dgrimes@ubuntuuserver:~$ cd /
dgrimes@ubuntuuserver:/$ cd ~/grimes/
dgrimes@ubuntuuserver:~/grimes$ cd /lib/apparmor/
dgrimes@ubuntuuserver:/lib/apparmor$ cd ..
dgrimes@ubuntuuserver:/lib$ _
```

Go to parent directory of *apparmor*

Go to subdirectory *lib/apparmor* of root directory

Moving around Linux system



A terminal window titled "Ubuntu 64-bit Server 14.04.5 2" with a status bar "To release your mouse press: Control-⌘". The terminal shows a user named "dgrimes" navigating through directories. The prompt is "dgrimes@ubuntuuserver:~/grimes\$". The user enters "cd -" and the prompt changes to "dgrimes@ubuntuuserver:/var/lib/apparmor/profiles\$". The user enters "cd -" again and the prompt changes to "dgrimes@ubuntuuserver:/home/dgrimes/grimes\$". The user enters "cd -" a third time and the prompt returns to "dgrimes@ubuntuuserver:~/grimes\$". A blue arrow points from the text "Go to previous directory" to the "cd -" command in the terminal.

```
dgrimes@ubuntuuserver:~/grimes$ cd -  
/var/lib/apparmor/profiles  
dgrimes@ubuntuuserver:/var/lib/apparmor/profiles$ cd -  
/home/dgrimes/grimes  
dgrimes@ubuntuuserver:~/grimes$
```

Go to previous directory

Become Root

- When you administer a system, it quickly becomes apparent how little you can do as a regular user.
- Configuration files require extra privileges to edit and every major service does as well.
- The root user is much like the Administrator user on a Windows system and has access to all of the files and services on the machine.

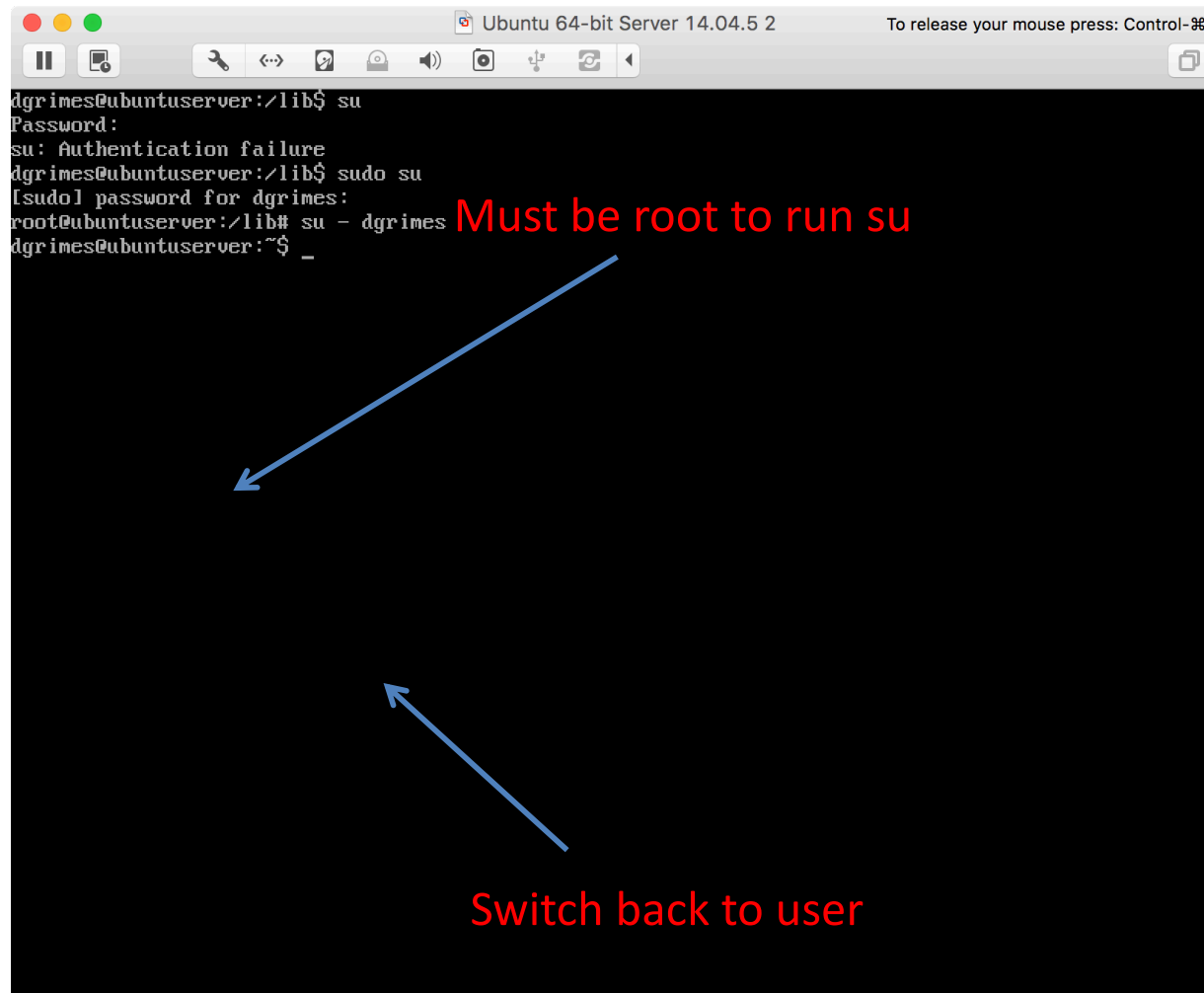
Become Root

- On some Linux systems the root user is like other users and has a password assigned to the account
- But on many
 - the root password is disabled.
 - Instead, you can log in as a regular user and then use the **sudo** command to gain root privileges.
 - Prefix commands with **sudo** or use **sudo su**

Running command as root or switching between root and user

<code>sudo</code>	“Super User DO”
<code>sudo <i>command</i></code>	Will ask for your password and perform the command
<code>su</code>	Switch to super user (i.e. root)
<code>su!</code>	But need root permissions to perform
<code>sudo su</code>	Enter your password
<code>su – <i>username</i></code>	Switch back to user account

Running command as root or switching between root and user



The image shows a terminal window titled "Ubuntu 64-bit Server 14.04.5 2". The terminal output is as follows:

```
dgrimes@ubuntuuserver:/lib$ su
Password:
su: Authentication failure
dgrimes@ubuntuuserver:/lib$ sudo su
[sudo] password for dgrimes:
root@ubuntuuserver:/lib# su - dgrimes
dgrimes@ubuntuuserver:~$ _
```

Two red annotations with blue arrows are present:

- The text "Must be root to run su" is positioned to the right of the terminal output, with a blue arrow pointing to the "su - dgrimes" command line.
- The text "Switch back to user" is positioned below the terminal output, with a blue arrow pointing to the prompt "dgrimes@ubuntuuserver:~\$ _".

Switching between root and user or running command as root

Why not always switch to root user or just login as root?

With great power comes great responsibility!
One mistake as root and could wipe the system.
sudo is safer option as forces user to type sudo
and to input password, reminding them that
their subsequent edits will be made as root

Viewing current structure

ls List names of files/subdirectories in the
 current directory

ls / List names of files/subdirectories in the
 root directory

ls ~/*dirname* List names of files/subdirectories of directory
 dirname in home

ls *sh List names of all files ending in “sh”

ls -l Give detailed list of files/subdirectories,
sorted by name

File type and permissions

-rwxrwx---- 1 dgrimes staff 48128 13 Apr 2012 f1.sh

drwx----- 23 dgrimes staff 782 11 Sep 09:46 dir1

Viewing current structure

```
dgrimes@ubuntu:~$ ls -l /
total 76
drwxr-xr-x  2 root root 4096 Sep 22 16:36 bin
drwxr-xr-x  3 root root 4096 Sep 23 16:44 boot
drwxr-xr-x 18 root root 4300 Sep 22 20:26 dev
drwxr-xr-x 89 root root 4096 Sep 23 16:23 etc
drwxr-xr-x  3 root root 4096 Sep 22 17:43 home
lrwxrwxrwx  1 root root   32 Sep 22 16:33 initrd.img -> boot/initrd.img-4.4.0-31-generic
drwxr-xr-x 21 root root 4096 Sep 22 16:36 lib
drwxr-xr-x  2 root root 4096 Sep 22 16:33 lib64
drwx----- 2 root root 16384 Sep 22 16:33 lost+found
drwxr-xr-x  3 root root 4096 Sep 22 16:33 media
```

Type and permissions

User/Owner

Group

Size in bytes of files

Number of
links/directories inside dir

Date last modified

Directory/file name

File/Directory Permissions

Restrict unauthorised access to files.

For example, there are core system files that you don't want any user to be able to edit (either through accident or malice) that could have serious consequences.

If you create a file or a directory, by default that file will have your username as owner and as group.

A group is a way of providing same permission settings on a file/directory to multiple users

Groups

- To share files or other resource with a small number of users
- Ease of user management
- Ease of user monitoring
- Group membership gives you or your user special access to files and directories or devices which are permitted to that group

Groups

Group names and their users listed in
`/etc/group`

View the groups that current user is part of:

```
$ groups
```

or

```
$ id -Gn
```

Recap: Linux and relevant commands

File system hierarchy?

Root then subdirectories, each may have its own subdirectories, etc.

/home/dgrimes/

Commands:

Change Directory?

Variations?

Path to current Working Directory?

LiSt contents of directory?

Run command as if you are root user who is just super

File/Directory Permissions

r – Reading a file

w – Writing / appending to / deleting a file/directory

x – Executing a program

Three categories for permissions: u = user, g=group, o=other

File/Directory Permissions

- rwx rw- r—

-/d/l/s file, directory, link, socket, etc

user read/write/execute

group read/write

other read only

Changing Ownership/Group

- As you administer a system, you will find that you often need to change a file's owner, group, or permissions.
- The `chown` and `chgrp` commands allow you to change the owner and group of a file, respectively, provided that your user has permission to change the ownership to begin with.
- So if I wanted to change the group of a file to be the staff group, I would type
`sudo chgrp staff filename`

Changing Permissions

chmod (*CH*ange access *MODe*)

chmod g=rw *filename* Set read/write
permissions for group

chmod u+x *filename* Change so that user can
execute the program

chmod o-wx *filename* Remove write and
execute

permissions for the user

chmod a+w *filename* Allow everyone write
permissions

chmod g=u *filename* Set permissions of group to be
same as user

Changing Permissions

Alternative numerical representation

$r = 4, w = 2, x = 1$

To set `rw-rw----`

`chmod 760 filename`

Viewing current structure

Number of other useful options for **ls** (can be combined)

-t/u Return a sorted list based on time of last access/
modification (Add an *r* to sort by oldest)

-R Recursively list items in each subdirectory

ls -lh Output in human readable format (K/M/G)

“ls -l” doesn’t give actual size of directory, it gives number of memory *blocks*.

Smallest memory block allowed on system is 4096 bytes, empty directory will still use 4096 bytes.

Viewing directory/file usage

du command: Disk Usage

du -h h for human readable like in ls

du -s for summary

Viewing directory/file usage

Sort du output in human readable format based on size

```
du -sh * | sort -h
```

Can help identify significant disk users, e.g. usr, var.

```
0      initrd.img
0      proc
0      sys
0      vmlinuz
4.0K   dev
4.0K   lib64
4.0K   mnt
4.0K   opt
4.0K   srv
4.0K   tmp
8.0K   media
16K    lost+found
24K    root
984K   run
2.3M   home
6.4M   etc
9.6M   bin
11M    sbin
38M    boot
352M   lib
497M   var
642M   usr
root@server1:~#
```


Editing Files

- On a Linux machine most programs store their configuration in text files.
- Linux has a number of text editors available, two popular default editors to choose from are **vi** and **nano**.
- Remember, will often need to use **sudo vi** for editing config files, etc as root only has permission to write to these files.

Creating/Editing Files and Directories

- Creating a file: *vi/nano/touch file_name*
- MaKing a DIRectory: *mkdir dir_name*
- MoVing a file: *mv file_name location*
- MoVing a directory: *mv dir_name location*
- CoPying a file: *cp file_name location*
- ReMoving a file: *rm filename*
- ReMoving a directory: *rm -r filename*

find command

Find all files in location matching criteria:

```
find location comparison_criteria  
search_term
```

Find all files that end in *php* in all subdirectories (recursive):

```
find -name "*php"
```

Find all files in directory *var* and its subdirectories that are empty

```
find /var/ -empty
```

Find all files beginning with *apache* but only go two subdirectory levels below

```
find /var/ -maxdepth 2 -name "apache*"
```

find command

Find all files/dirs not matching a criteria (-not) case-insensitive (iname), and redirect output to a file:

```
find /var/ -not -iname "apache*" > notapache.txt
```

Logical combinations of criteria (not and or)

Find all files/dirs ***either*** beginning with apache (case-insensitive) or ending in php:

```
find /var/ -iname "apache*" -o -name  
"*php"
```

Perform some function on found files, e.g. delete all:

```
find /tmp/ -type f -name "d*.txt" -exec rm  
-f {} \;
```

Note: **Ctrl+c** to stop the command as can generate large volume of output

Hidden files/directories

- Hidden items on Unix-like operating systems are easily distinguishable from regular (i.e., non-hidden) items because their names are prefixed by a period (i.e., a dot).
- Many OS hide objects to reduce the chances of users accidentally damaging or deleting critical system and configuration files.
- Hiding objects can also be useful for reducing visual clutter in directories, and thereby making it easier for users to locate desired files and subdirectories.
- Another reason to hide filesystem objects is to make them invisible to casual snoopers

Hidden files/directories

To show all files including hidden:

```
ls -A
```

Or just hidden files:

```
find / -name ".*"
```

- Intruders sometimes insert hidden files or directories containing malicious code, such as password cracking programs, into users' home directories, as their hidden nature makes them less obvious and more difficult to detect by casual inspection.
- It is a good idea for system administrators to occasionally check their systems for unusual or suspicious hidden files and directories

Checking Running Processes

- Check which programs are running on the system, what resources they are consuming & stopping them if necessary.
- `top`
 - Displays a lot of system information all at once
 - Sorts processes by how much CPU they use
- `ps`
 - Run with no arguments, it lists only processes in your current shell
- `ps -ef`
 - you will see volumes of information about all of the running processes on your system.

Killing Running Processes

- To kill a process:
 - Press the K key and then type in the PID you wish to kill, then hit “Enter” when prompted to kill with signal 15
- E.g. if I wanted to kill a program with a PID of 4023, I would type **kill 4023**.
- If that doesn't work:
 - Pass the -9 option
 - E.g. **Kill -9 4023**
 - Be sure to use regular **kill** first to give process a chance to clean up first

Don't forget.....

- The “man” command
- Used to display the user manual for Linux (and Unix) distributions
- Useful for exploring commands and options available

man chmod

CHMOD(1)

BSD General Commands Manual

CHMOD(1)

NAME

chmod -- change file modes or Access Control Lists

SYNOPSIS

```
chmod [-fv] [-R [-H | -L | -P]] mode file ...
chmod [-fv] [-R [-H | -L | -P]] [-a | +a | =a] ACE file ...
chmod [-fhv] [-R [-H | -L | -P]] [-E] file ...
chmod [-fhv] [-R [-H | -L | -P]] [-C] file ...
chmod [-fhv] [-R [-H | -L | -P]] [-N] file ...
```

DESCRIPTION

The **chmod** utility modifies the file mode bits of the listed files as specified by the mode operand. It may also be used to modify the Access Control Lists (ACLs) associated with the listed files.

The generic options are as follows:

- f** Do not display a diagnostic message if **chmod** could not modify the mode for file.
- H** If the **-R** option is specified, symbolic links on the command line are followed. (Symbolic links encountered in the tree traversal are not followed by default.)
- h** If the file is a symbolic link, change the mode of the link itself rather than the file that the link points to.
- L** If the **-R** option is specified, all symbolic links are followed.
- P** If the **-R** option is specified, no symbolic links are followed. This is the default.
- R** Change the modes of the file hierarchies rooted in the files instead of just the files themselves.
- v** Cause **chmod** to be verbose, showing filenames as the mode is modified. If the **-v** flag is specified more than once, the old and new modes of the file will also be printed, in both octal and symbolic notation.

The **-H**, **-L** and **-P** options are ignored unless the **-R** option is specified. In addition, these options override each other and the command's actions are determined by the last one specified.

Only the owner of a file or the super-user is permitted to change the mode of a file.

DIAGNOSTICS

The **chmod** utility exits 0 on success, and >0 if an error occurs.

MODES

Modes may be absolute or symbolic. An absolute mode is an octal number constructed from the sum of one or more of the following values:

Don't forget.....

- The “man” command
- Used to display the user manual for Linux (and Unix) distributions
- Useful for exploring commands and options available
- Another friend: *<tab>*
 - Auto-completion for commands / filenames, etc
 - *ls M<tab>* would autocomplete if there was only one file/directory in the current directory which begins with *M*. If more, then will display all that begin with *M*.
- Command history:
 - Stores most recent commands, use up arrow to go through them